A Governed Supervisory Execution Layer for LLM-Agent Plant Writes

Anonymous Authors

***Abstract*—Industrial large language model (LLM) agents require a traceable connection between task-level proposals, commanded controls, and process observations. This article presents a governed supervisory execution framework that connects these stages beneath a replaceable strategy layer. Semantic acquisition and control nodes connect agent identity to line–product–recipe–run context. Server-side channel profiles separate observation, model training, bounded exploration, and operational recipe work; capabilities and source-specific admission resolve execution authority. Required approvals govern actuation, and graded receipts distinguish transport acceptance from endpoint confirmation. Optimization records, batch receipts, and recipe versions retain the evidence appropriate to each lifecycle. Evaluation combines protocol and process missions, admission ablation, controlled confirmation challenges, nine regression executions totaling 782 checks, and a recipe-based injection-molding trajectory. Removing soft-window admission allowed six otherwise refused probes to execute, whereas removing readback left admission unchanged. A publish-only receipt retained its unverified grade despite successful transport acceptance. In one sequential recipe experiment, four approved pressure steps from 45 to 65 bar accompanied a weight change from 31.28 to 32.30 g with a fixed 6-s hold time, followed by retention of the accepted recipe. All process experiments used repository simulators. The results establish an inspectable execution chain linking task intent, authorization, control history, observed quality, and retained production parameters.**

***Index Terms*—Agent teams, human-in-the-loop, LLM agents, supervisory control, write governance.**

I. Introduction

LARGE language model (LLM) agents increasingly participate in industrial operation: they generate operator-reviewed OPC UA control sequences [**?**], invoke factory skills over tool protocols [**?**], coordinate shop-floor tasks [**?**], [**?**], and schedule a simulated robot-operated line [**?**]. Their usefulness depends on connecting task reasoning to the equipment and production context in which a proposal will be executed. That connection must express who may change a parameter, which product and recipe are active, what was actually commanded, and how the process responded.

Consider an injection-molding task that asks an agent to bring part weight into specification. The agent may consult recent measurements, diagnose insufficient packing, and propose higher holding pressure. Executing this proposal also requires a writable signal mapping, applicable parameter limits, an authorized production scope, and the required approval. The subsequent record must distinguish a pressure request, its endpoint confirmation, and the measured weight. These are different events with different evidence, even when one task spans the complete operation.

This information spans agent identity, industrial I/O, production management, and process history. Reconstructing it separately inside every agent application couples execution policy to the selected reasoning engine and obscures how an intervention relates to a running batch. We therefore address the following systems question: how can an industrial agent platform retain a common execution and evidence contract when the proposal strategy, equipment protocol, and production context change?

We present a governed supervisory execution framework connecting agent teams to semantic data-acquisition (DAQ) and write-control (DCW) nodes. A deterministic execution boundary sits beneath the replaceable strategy layer. The strategy acquires evidence, proposes a change, and interprets observations. The platform resolves authority, applies source-specific limits and required approval, dispatches commands, and grades confirmation. Line–product–recipe–run objects supply production context. Separate task, intervention, and batch records then connect the original intent to commanded controls and observed outputs.

The contribution is a common execution-and-evidence contract for industrial agent supervision. Its design and evaluation comprise:

1) a capability-based architecture joining channel identity, semantic industrial nodes, protocol drivers, and line–product–recipe–run context;
2) source-specific execution mechanisms combining admission, approval, graded command confirmation, attributed records, and separately governed compensation;
3) a current implementation that separates observation, model training, bounded optimization exploration, and operational recipe execution through server-side channel profiles; and
4) an evaluation linking commanded controls to observed outputs through admission ablation, protocol and confirmation drills, process missions, repeated regression runs, and an injection-molding recipe trajectory.

Section **??** connects team responsibilities and production objects to data, models, and proposals. Section **??** follows a proposal through execution and observation. Section **??** tests this chain in repository simulators, from admission checks to paired process and recipe trajectories. Section **??** interprets the evidence.

II. Related Work

A. Industrial Agents and Plant-Facing Action

Industrial agents supervise and reconfigure service-oriented cyber–physical systems [**?**]. Recent LLM work extends this

TABLE I
CLOSEST PLANT-FACING LLM SYSTEMS AND THE PROPOSED FRAMEWORK. ENTRIES SUMMARIZE THE MECHANISMS AND EVALUATION SETTINGS REPORTED BY EACH SOURCE.

System	Strategy and write interface	Authority/context emphasized	Reported evidence
MADE [**?**]	text strategy to structured OPC UA sequence	operator review and authorization before execution	40 control tasks; virtual and physical demonstrations
Köhle et al. [**?**]	module agents, MCP skills, OPC UA/MQTT	module/state-dependent tool availability	state cross-verification for silent faults; resolution rate in a simulated six-module plant
Waseem et al. [**?**]	LLM robot-assignment controller	prompt constraints in a simulated line	scheduling throughput
The platform	replaceable proposer above governed node/recipe execution	channel profiles, capabilities, product/recipe limits, run context	graded receipts, observations, attributed verdicts and batch records

direction to PLC generation and verification [**?**], instruction-guided control [**?**], digital-twin fault handling [**?**], and natural-language machine control over OPC UA [**?**]. These studies motivate a reusable boundary between an agent's proposal and its industrial execution.

The closest plant-facing systems emphasize complementary functions (Table **??**): operator-reviewed control-sequence generation with OPC UA execution [**?**], module-scoped skill dispatch [**?**], and LLM scheduling in a simulated production line [**?**]. Agent-allocation systems emphasize resource capability exploration and shop-floor coordination [**?**], [**?**]. Our comparison follows the authority and execution evidence connecting these strategies to production objects. The framework's approval and capability boundary implements the safety-aware human-in-the-loop decision-support perspective discussed in [**?**].

Köhle et al. also restrict tool availability by component state and cross-verify states to expose silent faults in a simulated six-module plant [**?**]. These mechanisms overlap with our admission and confirmation goals. The present design organizes that relationship around production-scoped capabilities, source-specific limits, and separate receipt/verdict semantics. A member's task, active recipe, commanded value, and observed response remain connected when the strategy or protocol changes.

B. Execution Contracts and Automation Practice

MCP standardizes tool discovery and calls with request-scoped authorization [**?**]; ISA-88 and ISA-95 supply recipe and operations concepts [**?**], [**?**]. The platform instantiates a reduced production model and adapts run-to-run control discipline [**?**] to agent-originated interventions. Software-agent constraints such as AgentSpec [**?**] and CaMeL's capability-based information-flow control [**?**] address tool behavior and data flow. Here those concerns meet semantic industrial quantities, active production context, and protocol-specific confirmation.

Electronic-record governance provides a complementary accountability perspective: FDA guidance discusses trustworthy records and audit trails [**?**], while IEC 62443-3-3 specifies industrial system-security requirements [**?**]. Command confirmation also has historical telecontrol precedents, including DNP3 select-before-operate [**?**]. The framework brings actor identity, production context, source-dependent admission, and retained confirmation evidence into one supervisory workflow. Control-oriented assurance addresses state evolution through shielding, control-barrier functions, and Simplex architectures [**?**], [**?**], [**?**]. Those approaches complement execution contracts that resolve which industrial operation is authorized and what evidence its execution produces. The systems question addressed here is how to preserve that contract across replaceable strategies, heterogeneous protocols, and versioned production context.

III. SYSTEM ARCHITECTURE

The platform connects task-level reasoning to production-scoped execution through three component groups (Fig. **??**). Team and strategy services maintain channels, members, tasks, messages, memory, and replaceable engine adapters. Governance services resolve capabilities, limits, approval, dispatch, confirmation, and intervention records. Industrial integration converts semantic DAQ/DCW operations into protocol I/O. These groups share line–product–recipe–run context and linked evidence stores. Registration services, a typed SDK, and REST, MCP, agent-to-agent, CLI, and console interfaces provide sanctioned extension points. Figure **??** groups these cooperating responsibilities rather than imposing one execution sequence on every operation.

The architecture separates three state objects with complementary roles. A task states the intended outcome and its responsible member. Production objects identify the configured definition and active execution context. Execution records state what was admitted, sent, confirmed, and observed. Shared identifiers join these objects for inspection. An agent engine can therefore be replaced without redefining a recipe or a driver's evidence contract. The project is distributed under Apache-2.0; repository links are withheld for anonymous review.

A. Semantic Nodes and Production Context

DAQ nodes expose engineering quantities with units, precision, timestamps, and quality metadata. Their contracts cover scalars, vector profiles, image frames, and mirrored manufacturing-execution-system (MES) records. MES integration supports mirrored acquisition and direct endpoint mappings. DCW nodes describe writable engineering quantities and their driver configuration. Drivers own serialization, transport handling, and available readback; platform services own authority and admission. Thus a proposal can name holding pressure in bar while the driver supplies the corresponding OPC UA, register, topic, or REST representation.

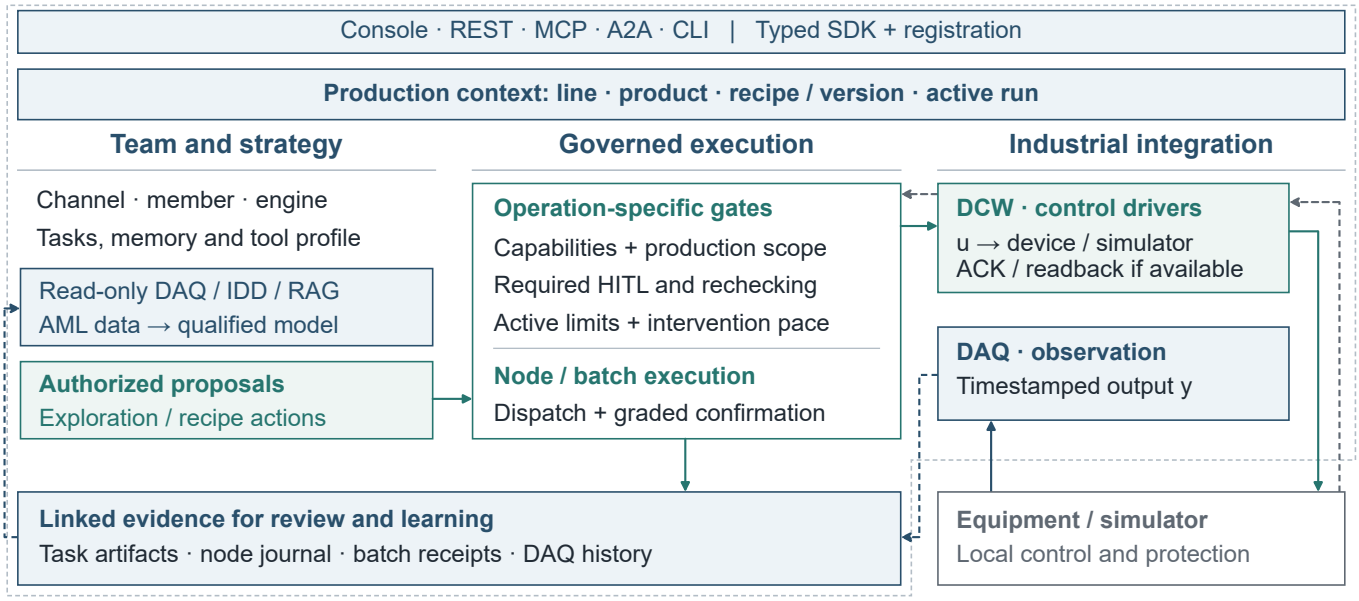


Fig. 1. Component architecture of the governed supervisory framework. Team and strategy services propose actions; governance services resolve authority and execution checks; industrial integration separates control u from acquisition y. Production context and linked evidence connect these responsibilities. Equipment protection and fast regulation operate outside the platform boundary.

Implemented protocol families include Modbus TCP, Modbus RTU-over-TCP, OPC UA, MQTT, and HTTP/REST, with MES REST mappings. Q1 exercises simulator writes over Modbus TCP, OPC UA, MQTT, and HTTP; RTU-over-TCP is acquisition-only. The admission ablation uses a separate mock driver. DAQ acquisition uses per-node cadences and a rotating sweep. Timestamped history supplies decisions, exports, and response queries independently of the agent's reasoning duration.

Lines associate equipment with an active product, recipe, and run. Recipe definitions are versioned; a running batch retains its execution context, so saving a definition and applying it are separate operations. This distinction supports a practical progression: investigate a candidate, observe its result, save the accepted parameterization, and apply the selected version to a run. Acquisition samples and dispatch receipts can then be interpreted against the production context in which they arose.

An onboarding blueprint declares devices, signal maps, units, roles, and initial setpoints. A differential ensure pass creates missing objects and reconciles declared configuration drift. Connecting another process changes this blueprint and its process-specific objective, guards, and proposal policy while reusing the execution services. This is the reuse boundary exercised across the process missions.

B. Capabilities and Channel Profiles

A channel organizes a job and its collaborating members; persisted capabilities authorize industrial operations. Node bindings define readable or controlled node scope and execution mode, while recipe capabilities define an authorized production scope. The server also restricts tools by channel profile. These two checks answer complementary questions: whether this class of job may invoke an operation, and whether this particular member may perform it on the selected production objects.

In the current v0.7.66 implementation, observation channels perform acquisition, analysis, and knowledge work, while Auto Modeling Lab (AML) training channels prepare datasets and models. Both have read-only plant access. AML-optimization channels invoke `optimization_explore` with control and target-scope validation. Operational channels use authorized recipe operations. Direct public `dcw_control` and `dcw_rollback` are closed; sanctioned platform workflows reach the internal governed node executor. Consequently, a written task instruction or retrieved document cannot expand the industrial tool surface.

This decomposition supports collaboration through explicit artifacts. An observation member can deliver a diagnostic report, a training member can deliver an evaluated model reference, and an optimization member can use authorized evidence to propose a bounded intervention. Each receiving member operates under its own channel profile and capability checks. Team coordination transfers information while execution authority remains attached to the actor and production scope.

C. Analysis, Models, and Candidate Proposals

The information path begins with timestamped acquisition and production context. A merged DAQ export aligns selected signals for industrial data diagnostics (IDD); diagnostic reports describe the observed operating regime. Retrieval-augmented generation (RAG) supplies relevant process knowledge and prior reports for interpretation. These outputs inform the strategy's choice of controls, targets, and stopping criteria. Write authority is resolved independently when the candidate reaches execution.

AML provides a separate data-to-model path. Its dataset builder selects line–product–recipe-scoped histories, cleans

samples by state and range, aligns them to a sampling grid, and constructs windows within runs. Run-based partitions and training-partition normalization produce a dataset manifest, tensors, and a cleaning report. These artifacts identify the controls, targets, features, runs, and transformations presented to training. The asynchronous production training route invokes Python, exports an Open Neural Network Exchange (ONNX) artifact, evaluates it, and applies gates before candidate registration. Registry stages distinguish candidate, shadow, production, and retired models; promotion carries its own approval workflow.

Model binding and eligibility checks govern model-assisted tools, including declared uncertainty, out-of-distribution, and physics information. Current lineage enforcement is primarily line-scoped. Each production training job retains its exported model, evaluation report, and gate decision as identifiable artifacts. Bounded exploration also operates without a model. A trained model, a candidate proposal, and an industrial command thus remain distinct outputs. The evaluation below focuses on execution contracts; model-assisted control–response is not evaluated.

D. Two Execution Lifecycles

Figure **??** separates the current runtime routes. Bounded optimization exploration validates the writable node and target ownership, then invokes the internal governed executor. It associates a single-factor control intervention with target observations. Operational recipe work uses `recipe_trial` for a candidate override without saving a version, `recipe_update` for a new definition, `recipe_apply` for dispatch, and `recipe_rollback` for version restoration. Restoring a definition and commanding the equipment are separately evidenced operations.

In Q6, an approved holding-pressure trial links its receipt to the next quality observation; a later recipe update retains the accepted parameters.

Persistence follows the route. A successful changed-value node write opens a node-optimization record before the observation wait. Bounded exploration then waits, queries the target, and saves a separate exploration summary. Recipe dispatch instead creates a batch run with per-parameter receipts; its caller performs the subsequent wait, observation, and decision. Saving an accepted parameterization requires a separate recipe update. These distinct records preserve the correspondence between an operation and its evidence. Section **??** identifies each experiment's tested interface.

IV. Governed Execution Mechanisms

The execution boundary converts a strategy's proposal into inspectable platform actions. It resolves the actor and production context and applies the authorization, admission, pacing, and approval checks required by that route. Driver dispatch produces command evidence; subsequent observation and interpretation complete the supervisory loop. Equipment execution and compensation retain separate outcomes.

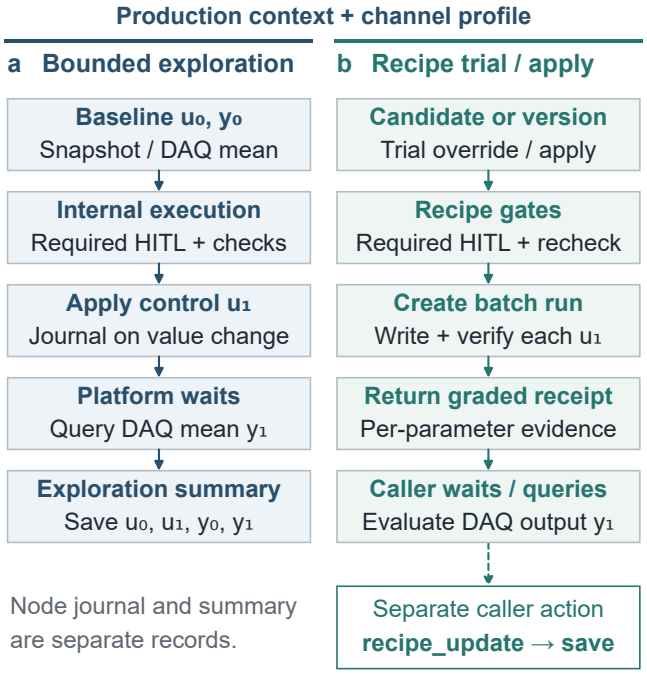


Fig. 2. Execution and evidence lifecycles. Bounded exploration opens a node record after a successful changed-value write, then waits and saves an exploration summary. Recipe dispatch returns batch and per-parameter receipts; its caller observes and decides. Retaining a recipe requires a separate update. Checks and approval follow each route's policy.

A. Authority, Admission, and Pacing

A request identifies the actor, operation, node or recipe, and production context. The server resolves its persisted capability and channel profile; authority comes from these records. For an online node intervention, the admitted interval is

$$\mathcal{I}_n^{\text{online}}(t) = \mathcal{H}_n \cap \mathcal{B}_n \cap \mathcal{P}_n(t) \cap \mathcal{R}_n(t), \tag{1}$$

where $\mathcal{H}_n$ is the hard engineering range and $\mathcal{B}_n$, $\mathcal{P}_n$, and $\mathcal{R}_n$ are baseline, active-product, and active-recipe limits. Missing optional limits impose no additional restriction. The interval in (**??**) is the value-admission input to the executor; an empty intersection refuses the request. The resolver identifies the excluding layer (Fig. **??**a), making the decision interpretable in production terms.

Admission follows the requested operation. Recipe dispatch checks $\mathcal{H}_n \cap \mathcal{B}_n \cap \mathcal{P}_n(t)$, omitting the existing recipe window when applying its replacement. This lets a new recipe carry its own setpoints while preserving equipment, baseline, and product bounds. Parameter admission constrains magnitudes. The process objective and coupled-variable guards are evaluated from subsequent observations.

Timing checks regulate supervisory interventions. Online node writes have an effective minimum spacing of 60 s. Recipe dispatch uses a persistent per-recipe interval anchored to approval; trials also use a configurable per-line cadence (default 300 s). Applicable checks run at proposal or execution according to the route; the recipe-operation interval can refuse a proposal before creating an approval card. Experimental settings are stated with each trial. Stored-setpoint hold heartbeats form a separate, default-off maintenance route that reapplies the stored value without fresh admission.

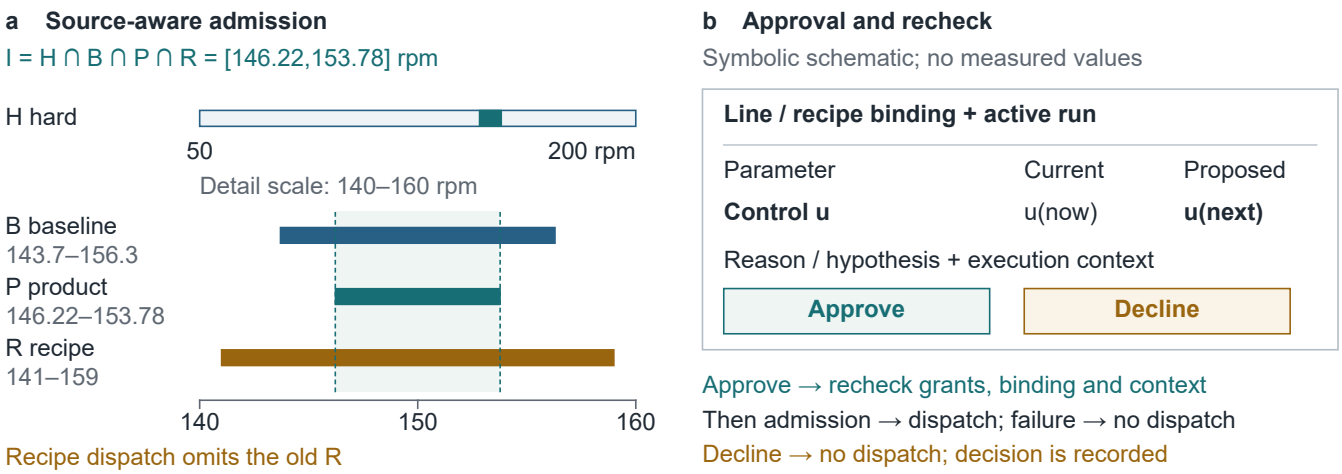


Fig. 3. Admission and approval boundaries. (a) The four-layer interval for an online node intervention; recipe dispatch omits the old recipe window. (b) Structured human approval separates the requested value and authority fields from display-only model reasoning. Approval is followed by execution checks and does not reserve equipment state.

B. Human Approval and Recipe Control

Manual execution bindings create an approval request carrying actor, node, candidate value, applicable interval, expiry, reasoning, and parameter bases. This structured object gives the approver the proposed operation alongside the governing context. The default timed wait expires after 180 s and refuses execution; a hold mode waits for an explicit decision. One pending request per member–node pair suppresses duplicates. The decision note returns to the proposing member, completing the interaction between strategy and approver.

Approval is bound to an action and followed by execution checks. The executor rechecks authorization and active limits, thereby accounting for capability or product changes during the wait. For `recipe_apply`, a changed definition requires a new proposal; an unchanged approval dispatches a copied parameter snapshot. The approved definition thus determines the values passed to batch dispatch.

A line-level recipe-dispatch master switch defaults to manual. It requires approval for otherwise approval-free recipe operations; changing it to automatic requires explicit administrator confirmation. The manual line mode also applies to internal exploration using an automatic recipe capability. Direct node bindings retain their own execution mode and admission checks. Agents use these policies but cannot change the master switch. Approval records workflow authorization; process protection retains its independently engineered control-system role.

The proposer owns the candidate and interpretation, the authenticated approver authorizes the pending action, and the verdict issuer owns the recorded decision. Receipts and operations logs identify outcomes needing operator follow-up. Deployment assigns that responsibility and the line-specific approval roles; closing a record is a lifecycle operation rather than proof of equipment recovery.

C. Graded Command Confirmation

Command evidence has two levels: the driver's acceptance ACK and the batch verifier's terminal verdict. The driver assigns `readback-verified`, `transport-ack`, or `unverified`. A matching post-command read supports the first; a transport or MES application receipt supports the second. The batch verifier assigns `verified`, `unverified`, or `failed`. It accepts an existing matching readback or performs up to three rereads, 2 s apart. A matching reread upgrades the final ACK to `readback-verified`; exhausted confirmation attempts produce an unverified verdict, and command failure produces a failed verdict. MES acceptance without a read mapping follows the unverified outcome. Write history retains the initial ACK, while the batch result stores the final ACK and terminal verdict.

Batch dispatch records each parameter's result and summarizes verified, unverified, and failed counts. This granularity preserves a verified parameter even when another parameter's command fails. A readback-verified value describes the endpoint's returned setting; target attainment is evaluated from process observations. Section **??** tests the distinction with a publish-only endpoint whose `ok=true` broker receipt is transport acceptance and whose terminal confirmation remains unverified.

D. Observation, Verdict, and Compensation

Bounded exploration couples an intervention with a target summary. It captures the starting control before asynchronous execution and averages 1-s buckets over a trailing 45-s target window. A successful changed-value write opens the node record. The exploration workflow waits 10–300 s (default 45 s) and queries another trailing 45-s window. These are query windows relative to their query times: approval can separate the baseline from actuation, and a 10-s wait can leave about 35 s of pre-write observations in the later window. The receipt and target statistics are returned separately for interpretation of this single-factor probe. The exploration summary is saved after this observation; refused or expired approval instead produces a rejected summary without actuation or post-write sampling.

The governed node lifecycle accepts attributed `keep`, `rollback`, or `uncertain` verdicts with the caller's inter-

pretation and queried window. Compensation requests a new command under its applicable admission policy and retains its own outcome, including partial completion. In the archived experiments, a harness also compares recipe value, endpoint readback, and an acquisition-store sample. This three-way check measures agreement between representations sharing an endpoint.

A record backstop supplies monitoring while an optimization record remains open. After 120 s it polls every 30 s; three cumulative samples outside the recipe monitoring window trigger a system-attributed rollback toward the preceding value. Its window has a monitoring role separate from the admission interval and mission target. Available samples determine breach counts; missing samples contribute no breach. A caller's keep verdict closes the record and its monitoring. When retained rollback records reach the configured limit, further actuation follows the approval policy.

Attribution joins actor, production context, receipt, verdict, and recipe/run records. The write audit deduplicates the same value on the same node within 10 s, while batch records and optimization journals retain their route-specific lifecycles. Together these records let an investigator trace the command, the evidence available to its evaluator, and the parameterization retained for subsequent work.

V. Experimental Evaluation

The evaluation follows the execution chain from interface reuse to process evidence. Q1 tests protocol closure and the distinct roles of admission and confirmation. Q2–Q3 replace the proposal strategy with agent and controller workflows. Q4 pairs controls and outputs across four processes. Q5 tests repeated deployment execution, timing, and confirmation semantics. Q6 follows a complete operational recipe trajectory from baseline observation to a saved version. All process experiments use repository simulators.

A. Evidence Units and Experimental Procedure

Q1–Q4 use the seed-42 integrated archive `20260921184204-10lc`, its raw scenario records, and companion ablation run `20260921181234-e1lite`. The cast-film reference preset has an author-defined objective J combining thickness conformance, defects, energy, and throughput. These historical missions use the direct-node interface. Scripted policies occupy the proposal slot except for two live engine tasks in Q2. Q3 additionally uses seeds 43–44. Q5 uses nine deployment-suite executions from October 2026; Q6 uses the October 18 recipe archive. The archives identify each tested interface and its own evidence unit. Separately, 65 current channel/profile and composition tests pass in temporary storage with mock I/O, covering approval outcomes, starting-control capture, and approved-recipe snapshot consistency. Q6 exercises the recipe route; these evidence sets are not pooled as one interface experiment.

The process missions follow six stages. First, provision the semantic nodes, their signal mappings and limits, and a baseline recipe in the active production context. Second, acquire the initial controls and target observations to establish the starting state. Third, submit a bounded candidate under the declared capability, binding, and approval policy. Fourth, retain the dispatch receipt and available readback, preserving each parameter's verification result. Fifth, wait the scenario's declared response interval and query the relevant DAQ quantities. Sixth, evaluate the observed target and guards, then continue, compensate, or stop. A retained parameterization can subsequently become a recipe version.

This ordering produces three complementary result objects. The requested control sequence u records the intervention, the endpoint confirmation records command evidence, and the target sequence y records process observations. The task verdict joins them under the declared acceptance criteria. Figures **??** and **??** display u and y together so that a target change can be inspected alongside the corresponding control history; these plots validate evidence linkage and route behavior. An evidence unit is a run or a sequential trajectory; several writes within it are sequential observations rather than independent trials.

The integrated suite has 20 phases spanning commissioning, protocol I/O, governance, agent missions, controller reuse, and concurrent processes. The deployment pipeline uses S0 prechecks, S1 APIs, S2 acquisition/MES, S3 negative paths, S4 closed-loop execution, S5 fine-tuning, S6 diagnostics, and S7 report generation. Checks are author-defined regression assertions. Reported quantities are rejected probes, per-parameter confirmation grades, recipe/readback/mirror deviations, process-target margins, and timestamp spans. At approximately six writes per protocol line, latency is reported as a median and observed maximum rather than a tail percentile.

B. Q1: Protocol Closure and Admission Ablation

The suite acquired 55 samples over five protocol families and issued governed writes over four; the RTU-over-TCP line was acquisition-only. Median write latencies ranged from 19.3–47.8 ms, with observed maxima 20.1–248.9 ms. The four writable lines accepted legal writes and rejected 24/24 out-of-window probes. A configuration-only second preset added 9/9 rejected probes on three further writable lines. These results exercise the same resolver against different signal maps.

Table **??** isolates admission from confirmation in a mock-driver API fixture. Each arm replayed six attack classes in three deterministic repetitions (18 attacks per arm); separate boundary, legal, and timed probes are excluded from that denominator. Removing the soft-window interlock allowed six soft-window probes to execute, while the hard-range probes remained refused. Archived outcomes and journal records link the refusals and executed commands to their configured arms. Removing readback did not alter admission. Confirmation can therefore check an execution outcome, but cannot replace a pre-write gate. A frozen-measurement drill further separated command and process evidence: the setpoint and readback agreed, yet the monitoring quantity remained outside its window and triggered backstop rollback after 130.2 s. A manual-binding drill suspended execution until harness-issued approval (461 ms mediation span).

TABLE II
MOCK-DRIVER API ABLATION: SIX ATTACK CLASSES, THREE DETERMINISTIC PROPERTY-TEST REPETITIONS PER ARM (18 ATTACKS, EXCLUDING SEPARATE BOUNDARY PROBES). FOUR HARD-RANGE CLASSES ARE REFUSED IN EVERY ARM; TWO SOFT-WINDOW CLASSES CHANGE OUTCOME.

Arm	Soft window	Readback	Refused	Executed
full	✓	✓	18/18	0
no-readback	✓	—	18/18	0
no-interlock	—	✓	12/18	6
ungated	—	—	12/18	6

C. Q2–Q3: Strategy Replacement and Controller Handling

The archived team workflow linked a parent task, a bound worker, a command, a verdict, and its journal. On the single-node cast-film mission, the policy changed the heater from 200 to 204.704 °C; the observation settled at 204.70 °C within the declared target. The task reached `COMPLETED`. The BOPET mission expanded this workflow to nine devices and 49 signals, with three manipulated nodes (Section **??**).

Two live engine tasks (omp adapter, glm-5.3-flash) occupied the same proposal slot on a scratch line. The archive records a bounded write, a goal-driven descent to 51.7 μm for a $52.0 \pm 0.8\,\mu$m target, and a recipe save. The original contemporaneous omp logs and checked run outcomes trace proposal, governed action, observations, and retained recipe. The reported 51.7 μm is the run-summary terminal value; the log's 52.02 μm window mean is a different aggregation.

A deterministic controller used the dimensionless cast-film objective

$$J = 55J_{\text{thick}} + 25J_{\text{quality}} + 8J_{\text{energy}} + 7J_{\text{throughput}}, \tag{2}$$

whose components score thickness conformance, product quality, and control-based energy and throughput proxies. The latter two terms use screw speed and line speed, rather than independently metered energy and production output. The functions in (**??**), constraints, grid, and replay settings are specified in the accompanying `REPRODUCIBILITY.md`. It made two governed writes per seed (42–44), without rejection. It raised J from approximately 67.2 to $J/J^* = 0.965$–0.972, with $J^* = 89.894$ from an admissible setpoint grid. This finite restricted grid supplies a reference normalization, not a global continuous optimum. The result links the controller's native objective to admissible commands and endpoint evidence, exercising a conventional strategy through the same execution services.

D. Q4: Paired Controls and Outputs Across Processes

The concurrent campaign provides one seed-42 trajectory per process. Figure **??** pairs archived manipulated controls u with the corresponding response y rather than showing response alone. All four terminal attainment checks passed; two lay near a target boundary.

Injection molding used two changes: hold pressure $45 \rightarrow 58.2$ bar and hold time $6 \rightarrow 10.64$ s. Part weight rose from 31.25 to 32.17 g for a 32.5 ± 0.35 g window, only 0.02 g above its lower limit. BOPET changed casting speed $32 \rightarrow 33.8$ m/min, fast-roll speed $118 \rightarrow 120.3$ m/min, and rail width $3000 \rightarrow 3035$ mm. Thickness fell from 28.10 to 25.66 μm for a $25.0 \pm 0.7\,\mu$m target, $0.04\,\mu$m inside the upper limit. Its three changes each have a recorded keep verdict and journal identifier.

TABLE III
DEPLOYMENT-SUITE ASSERTIONS ACROSS NINE SEQUENTIAL EXECUTIONS; CHECKS ARE DETERMINISTIC ASSERTIONS, NOT INDEPENDENT PLANT SAMPLES.

Pass	Test surface	Runs	Checks
1	baseline S0–S7	3	237
2	+ grading and master switch	3	273
3	hardened regression surface	3	272
Total	zero failed checks	9	782

Wastewater treatment required 15 writes across several nodes. Dissolved oxygen rose from 0.21 to 3.50 mg/L, within 3.4 ± 0.6 mg/L. Some admissible blower proposals reduced compliance margin and were reverted. Terminal guards remained within their declared limits, although pH moved toward its lower bound (6.59 to 6.38; limit 6.0). Dissolved oxygen, total phosphorus, and pH are different quantities and are not concatenated into one output trajectory.

Annealing combined furnace-temperature changes with line-speed increases from 140 to 164 m/min. The recorded hardness trajectory moved from 134.52 to 99.84 HV; the separate terminal measurement was 100.08 HV, within 95 ± 6 HV. A 176 m/min attempt produced 101.3 HV and was reverted. The first temperature step has no optimization record identifier in the archive, which bounds record-level traceability. Together, the four missions exercise configuration-based process reuse and retain the distinction between a candidate command, a reverted candidate, and a terminal observation.

E. Q5: Regression, Timing, and Confirmation Semantics

The deployment instance contained 34 provisioned lines, 111–113 channels, 67 online DAQ nodes, 77 write nodes, 52 recipes, and 385 bindings, with 17 lines running. These objects coexisted on one simulator-backed host. Nine pipeline executions passed 782 checks (Table **??**).

The injection benchmark followed observation, proposal, pacing gate, approval, batch dispatch, and recipe/readback/mirror comparison. The three sources agreed at the endpoint's recording resolution (maximum deviation 0.0 bar; tolerances 0.05 bar for device readback and 0.51 bar for the bucketed mirror). This verifies setpoint consistency rather than process convergence. In the illustrated S4 run, the trial cadence refused the initial proposal before approval; the reproposal occurred 69 s later and observation-to-verification took 96.105 s. S5's wait-to-restoration span was approximately 66 s (Fig. **??**); the archived process result remained `converged=false`. Across the campaign, approval-to-dispatch spans were 6–9 ms. These spans measure server/harness execution between the recorded events.

The seven-leg fault suite tested the interpretation of receipts. A mock acceptance was promoted by reread; a broker outage produced one failed parameter while its healthy sibling verified; and a publish-only endpoint returned a broker receipt yet remained unverified after three attempts. Two parameters

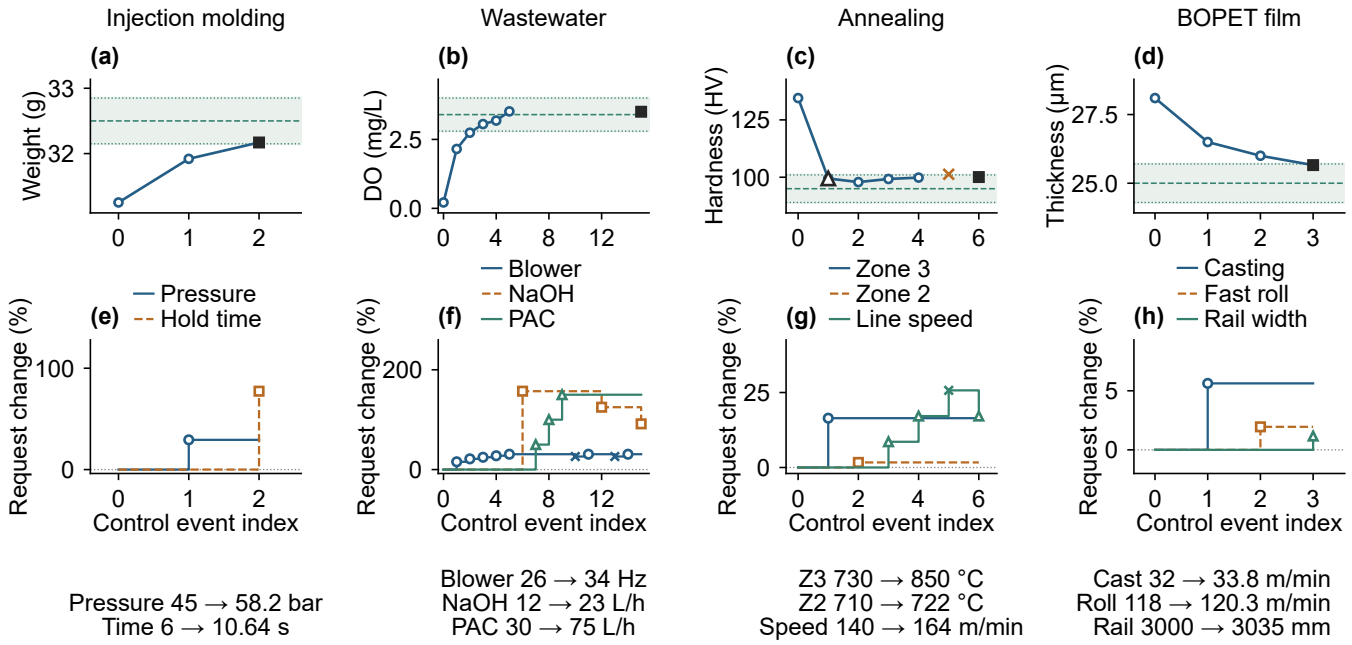


Fig. 4. Historical scripted missions (seed 42, one trajectory per process). (a–d) Weight, dissolved oxygen (DO), hardness, and film thickness with their target windows. (e–h) Requested controls as percentage change from native baselines: injection pressure 45 bar/time 6 s; wastewater blower 26 Hz/NaOH 12 L/h/PAC 30 L/h; annealing zone 3 730°C/zone 2 710°C/speed 140 m/min; BOPET casting 32 m/min/fast roll 118 m/min/rail 3000 mm. Lines connect event indices, not physical time; unchanged requested values carry forward. Crosses mark reverted candidates and squares distinct terminal observations. DO excludes pH/TP events; annealing's first step lacks an optimization record.

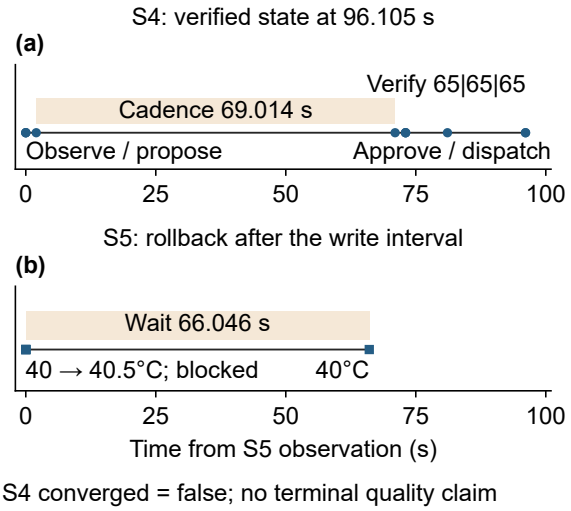


Fig. 5. Archived governance timing from event timestamps. (a) S4 takes 96.105 s from observation to set/device/mirror agreement at 65 bar, including a 69.014-s cadence wait. (b) S5 starts 96.129 s after the S4 observation, applies 40 → 40.5 °C, blocks a further write, waits 66.046 s, and restores 40°C. Each row has its own elapsed-time origin. Scripted approval and command verification do not establish quality convergence; the S4 verdict is `converged=false`.

were promoted from transport acknowledgment to readback verification. Manual-to-automatic mode changes required confirmation, and a manual-mode recipe release queued approval before six parameters verified. API and documentation checks completed the suite. MES application-receipt semantics are covered by route tests, whereas the campaign used MES only in read mode. The suite therefore checks that receipt grades follow the available endpoint evidence under each deliberately injected condition.

F. Q6: Recent Recipe-Based Injection Trajectory

The October 18 recipe archive (R6 in the accompanying reproducibility notes) pairs recipe-path controls, receipts, and process observations. The replay entry is `scripts/e2e-injection-recipe-closedloop.mjs`. The setup used OPC UA setpoints and HTTP quality acquisition against the injection simulator. A live omp/deepseek-flash worker separately diagnosed insufficient packing. The archived intervention sequence and approvals were scripted, allowing the candidate, receipt, wait, observation, and recipe-update sequence to be inspected explicitly.

Starting from 45 bar and a fixed 6-s hold time, the script proposed 5-bar pressure increments. Each step called `recipe_trial`, answered the pending approval card, and retained the batch receipt. After 65 s, it queried platform control-node values and DAQ outputs. It stopped when weight was in $[32.15, 32.85]$ g, flash was at most 0.4%, and sink mark was at most 1.5%, or after at most six pressure proposals. The replay also defines overshoot compensation and a hold-time fallback. The archived successful run used four pressure proposals and no hold-time change. The harness set `AW_TRIAL_INTERVAL_MS=0`, disabling the per-line trial cadence, and issued `emergency=true` trials to bypass the separate recipe-operation interval. Approval, admission, and command confirmation remained active. This configuration tests the execution chain with experimental pacing. An earlier archived attempt exercised the default 300-s cadence: it refused a premature trial and the target was not attained in that attempt.

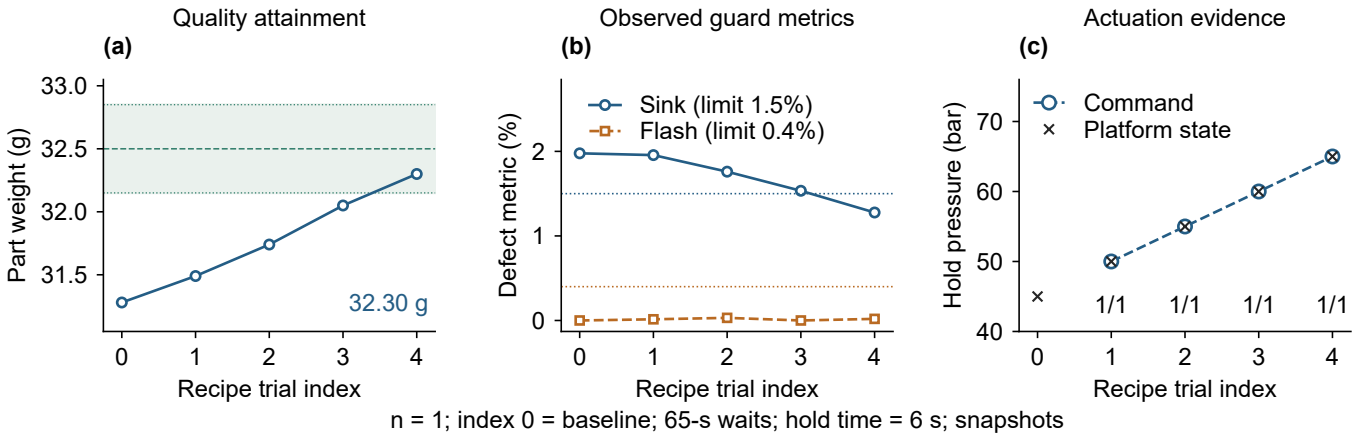


Fig. 6. October 18 recipe-path injection case study ($n = 1$ simulator trajectory). (a) Weight with the 32.15–32.85-g band and 32.5-g center. (b) Sink and flash snapshots with 1.5% and 0.4% limits. (c) Requested pressure and platform-node state after each 65-s wait; overlapping symbols indicate equality, and each trial receipt reports 1/1 parameter verified. Hold time remains 6 s. Points are discrete snapshots, not physical PLC traces; the harness supplied approvals.

Figure **??** shows the complete baseline-to-terminal sequence. Pressure advanced $45 \rightarrow 50 \rightarrow 55 \rightarrow 60 \rightarrow 65$ bar; weight increased $31.28 \rightarrow 31.49 \rightarrow 31.74 \rightarrow 32.05 \rightarrow 32.30$ g, and sink mark decreased from 1.977% to 1.277%. Final flash was 0.019%. Each trial receipt reported one verified parameter out of one. The displayed controls are platform-node snapshots, while the receipt supplies the archived verification grade. After the final check, `recipe_update` saved version 2 with 65 bar/6 s. The recipe log contains four trials and the update, while the optimization journal is empty, as expected for this lifecycle.

The paired plot explains why the script continued and where it stopped. At 60 bar, the 32.05-g observation remained below the 32.15-g lower acceptance limit, so the next 5-bar candidate was required by the declared rule. At 65 bar, the 32.30-g observation entered the weight band, and the 1.277% sink-mark and 0.019% flash observations satisfied their respective limits. Holding time remained 6 s throughout, making pressure the sole changed control in this trace. The final recipe update records the accepted 65 bar/6 s parameterization; it is distinct from both the four trial receipts and the observed quality values.

The quality assertion allowed a missing melt-temperature value, and the freshness check used the newest timestamp among available sensors. The reported pass therefore covers the three observed quality quantities; temperature and individual-sensor freshness remain outside that assertion. The duplicate archive adds no repeat. A separate later check started at 65 bar, 32.29 g, and 1.385% sink mark; one 70-bar trial ended at 32.55 g and 0.976%. It is a new initial state and is not appended to the plotted trace. Q6 connects baseline evidence, approved control changes, observed quality, and the retained recipe through one inspectable operational loop.

VI. Engineering Interpretation

The results support an integrated systems contribution: industrial agent execution can preserve authority, production context, command confirmation, and process observations as distinct, inspectable evidence. Admission ablation explains the contribution of the pre-write interval, the publish-only challenge tests interpretation of command receipts, and the paired trajectories connect interventions to observed targets. Their combination makes an agent's completed task interpretable in terms of the industrial operations it performed.

This evidence structure also explains the division between strategy and platform. An LLM, scripted policy, or deterministic controller supplies a candidate through its authorized interface. The platform applies the execution contract and returns evidence with stable semantics. Likewise, process reuse changes the signal map, objective, and guard definitions while retaining production context and execution records. Q6 closes this chain at the operational level: the retained recipe version follows observed acceptance of an approved trial sequence.

Human participation is represented through explicit authority and attributed decisions. This supports inspection of what an approver authorized and what evidence the proposer later used. Automation research distinguishes such workflow structure from operator comprehension, complacency, and allocation of responsibility [**?**], [**?**], [**?**]. The server-side boundary resolves execution authority independently of prompts and retrieved documents [**?**], [**?**]. Deployment combines this boundary with credential management, endpoint security, and IT/OT separation [**?**]. The supervisory layer coexists with independently engineered PLC interlocks and safety instrumented systems; IEC 61511 governs the latter's process-safety lifecycle [**?**].

Experimental scope. The process evidence uses accelerated simulators, scripted approvals, and sequential trajectories. Q1–Q4 cover the historical direct-node interface; current profile and composition tests cover software behavior; Q6 covers one recipe trajectory. A complete current AML-exploration control–response trial is not archived. Q6's observed guards, snapshots, and cadence settings are specified in Section **??**. These evidence units establish the exercised mechanisms without estimating comparative optimization benefit, fault frequency, human performance, hard real-time behavior, or physical transfer. Matched initial states, repeated independent runs, and a fixed-policy or SCADA comparator would support those further estimates. Established process benchmarks such

as Tennessee Eastman [**?**] provide an additional setting for that evaluation.

VII. Conclusion

The framework connects industrial agent teams to production-scoped, governed execution while keeping proposal strategy replaceable. Semantic nodes, capability/profile enforcement, source-specific admission, approval, graded confirmation, and distinct node/recipe records form an inspectable chain from a task to commanded controls, observed outputs, and retained production parameters. Protocol and admission experiments, controller and process missions, repeated regression execution, and the paired injection-molding trajectory exercise complementary parts of that chain. The contribution is a common execution and evidence contract through which industrial agent reasoning can be connected to accountable supervisory operation.

Data availability and AI assistance. Run archives, original Q2 engine logs, replay entries, configuration notes, and claim-to-source mappings accompany the separate anonymous experimental evidence bundle. The manuscript source package contains plotting sources and figure mappings. An OpenAI language model assisted with manuscript organization, language revision, and plotting-code preparation. The authors remain responsible for the final submission.

References

[1] K.-C. Huang, H.-T. Lin, C.-H. Lin, and M.-T. Huang, "Large language model integration in industrial cyber-physical systems: A multi-modal architecture for automated control generation," in *Proc. Int. Conf. on Control, Automation and Diagnosis (ICCAD)*, 2026, pp. 1–6, doi: